



Міністерство освіти і науки України
Національний технічний університет України
“Київський політехнічний інститут імені Ігоря Сікорського”
Факультет інформатики та обчислювальної техніки
Кафедра інформаційні систем та технологій

Лабораторна робота № 3
із дисципліни «Розробка мобільних застосунків під Android»
Тема: «ДОСЛІДЖЕННЯ СПОСОБІВ ЗБЕРЕЖЕННЯ ДАНИХ»

Виконав

Студент групи ІА-31:

Губар Б. О.

Перевірив:

Орленко С. П.

Київ 2026

1. Мета: дослідити способи збереження даних (база даних, файлова система, тощо) та отримати практичні навички щодо використання сховищ даних.

2. Завдання:

Написати програму під платформу Андроїд, яка доповнює програму з ЛР №2 роботою зі сховищами. При натисканні «ОК» здійснюється запис результату до сховища (файл) і користувач інформується повідомленням щодо успішності запису. Інтерфейс доповнено кнопкою «Відкрити», яка переводить на іншу Діяльність для відображення вмісту сховища. Якщо сховище пусте — відображається відповідне повідомлення .

5.	Вікно замовлення квітів містить: текстове поле, дві групи опцій (колір, діапазон цін), тобто радіо-батонів та кнопку «ОК». Вивести інформацію щодо замовлення при натисканні на кнопку «ОК» у деяке текстове поле.
----	--

3. Хід роботи

Оновлення InputFragment.kt: У логіку додано використання методу `openFileOutput` з параметром `Context.MODE_APPEND`. Це дозволяє дописувати нові замовлення в кінець існуючого файлу `orders_history.txt` без перезапису попередніх даних. Також реалізовано перехід на нову діяльність за допомогою `Intent`.

```
package com.example.lab1_flowers

import android.content.Context
import android.content.Intent
import android.os.Bundle
import android.view.LayoutInflater
import android.view.View
import android.view.ViewGroup
import android.widget.Button
import android.widget.EditText
import android.widget.RadioButton
import android.widget.RadioGroup
import android.widget.Toast
import androidx.fragment.app.Fragment
import androidx.lifecycle.ViewModelProvider
import java.io.FileOutputStreamStream

class InputFragment : Fragment() {
    private lateinit var viewModel: SharedViewModel

    override fun onCreateView(
        inflater: LayoutInflater, container: ViewGroup?,
        savedInstanceState: Bundle?
    ): View? {
        val view = inflater.inflate(R.layout.fragment_input, container, false)

        viewModel = ViewModelProvider(requireActivity())[SharedViewModel::class.java]

        val editTextInput = view.findViewById<EditText>(R.id.editTextInput)
        val radioGroupColor = view.findViewById<RadioGroup>(R.id.radioGroupColor)
        val radioGroupPrice = view.findViewById<RadioGroup>(R.id.radioGroupPrice)
        val buttonOk = view.findViewById<Button>(R.id.buttonOk)
        val buttonOpenHistory = view.findViewById<Button>(R.id.buttonOpenHistory)

        buttonOk.setOnClickListener {
            val name = editTextInput.text.toString().trim()
```

```

        val selectedColorId = radioGroupColor.checkedRadioButtonId
        val selectedPriceId = radioGroupPrice.checkedRadioButtonId

        if (name.isEmpty() || selectedColorId == -1 || selectedPriceId == -1) {
            Toast.makeText(requireContext(), "Будь ласка, заповніть всі поля!",
Toast.LENGTH_SHORT).show()
        } else {
            val colorRadioButton = view.findViewById<RadioButton>(selectedColorId)
            val priceRadioButton = view.findViewById<RadioButton>(selectedPriceId)

            val result = "Замовлення оформлено!\nЗамовник: $name\nКолір квітів:
${colorRadioButton.text}\nКатегорія: ${priceRadioButton.text}"

            viewModel.orderResult.value = result

            try {
                val fos: FileOutputStream =
requireContext().openFileOutput("orders_history.txt", Context.MODE_APPEND)
                val textToSave = "$result\n-----\n"
                fos.write(textToSave.toByteArray())
                fos.close()

                Toast.makeText(requireContext(), "Дані успішно збережено у файл!",
Toast.LENGTH_SHORT).show()
            } catch (e: Exception) {
                e.printStackTrace()
                Toast.makeText(requireContext(), "Помилка при збереженні даних!",
Toast.LENGTH_SHORT).show()
            }
        }
    }

    buttonOpenHistory.setOnClickListener {
        val intent = Intent(requireContext(), HistoryActivity::class.java)
        startActivity(intent)
    }

    viewModel.clearFormEvent.observe(viewLifecycleOwner) { shouldClear ->
        if (shouldClear) {
            editTextInput.text.clear()
            radioGroupColor.clearCheck()
            radioGroupPrice.clearCheck()
            viewModel.clearFormEvent.value = false
        }
    }

    return view
}
}

```

Оновлення UI (fragment_result.xml): Для покращення користувацького досвіду (UX), кнопку 'Cancel' було перейменовано на 'Нове замовлення', оскільки дані вже фізично записані в пам'ять, і кнопка виконує лише функцію очищення UI.

```

<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:orientation="vertical"
    android:padding="16dp">

    <androidx.cardview.widget.CardView

```

```

        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        app:cardBackgroundColor="#E8F5E9"
        app:cardCornerRadius="12dp"
        app:cardElevation="2dp">

        <TextView
            android:id="@+id/textViewResult"
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:padding="16dp"
            android:text="Тут з'явиться інформація про замовлення"
            android:textColor="#2E7D32"
            android:textSize="16sp"
            android:textStyle="italic" />
    </androidx.cardview.widget.CardView>

    <Button
        android:id="@+id/buttonCancel"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:layout_marginTop="8dp"
        android:backgroundTint="#D32F2F"
        android:text="Нове замовлення"
        android:textColor="#FFFFFF" />

</LinearLayout>

```

HistoryActivity.kt: Створено новий екран для читання даних. Використовується `openFileInput` разом із `BufferedReader` для порядкового зчитування вмісту файлу. Реалізовано обробку винятків (`try-catch`): якщо файл ще не створено або він порожній, користувачу виводиться повідомлення про відсутність даних.

```

package com.example.lab1_flowers

import android.os.Bundle
import android.widget.Button
import android.widget.TextView
import androidx.appcompat.app.AppCompatActivity
import java.io.BufferedReader
import java.io.FileInputStream
import java.io.InputStreamReader

class HistoryActivity : AppCompatActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_history)

        val textViewHistory = findViewById<TextView>(R.id.textViewHistory)
        val buttonCloseHistory = findViewById<Button>(R.id.buttonCloseHistory)

        try {
            val fis: FileInputStream = openFileInput("orders_history.txt")
            val isr = InputStreamReader(fis)
            val bufferedReader = BufferedReader(isr)
            val stringBuilder = java.lang.StringBuilder()
            var text: String?

            while (bufferedReader.readLine().also { text = it } != null) {
                stringBuilder.append(text).append("\n")
            }

            textViewHistory.text = stringBuilder.toString()
        } catch (e: Exception) {
            textViewHistory.text = "Даних немає"
        }

        buttonCloseHistory.setOnClickListener {
            finish()
        }
    }
}

```

```

    }
    fis.close()

    val historyText = stringBuilder.toString()

    if (historyText.trim().isEmpty()) {
        textViewHistory.text = "Дані відсутні (сховище пусте)."
    } else {
        textViewHistory.text = historyText
    }

} catch (e: Exception) {
    textViewHistory.text = "Дані відсутні (сховище пусте)."
}

buttonCloseHistory.setOnClickListener {
    finish()
}
}
}

```

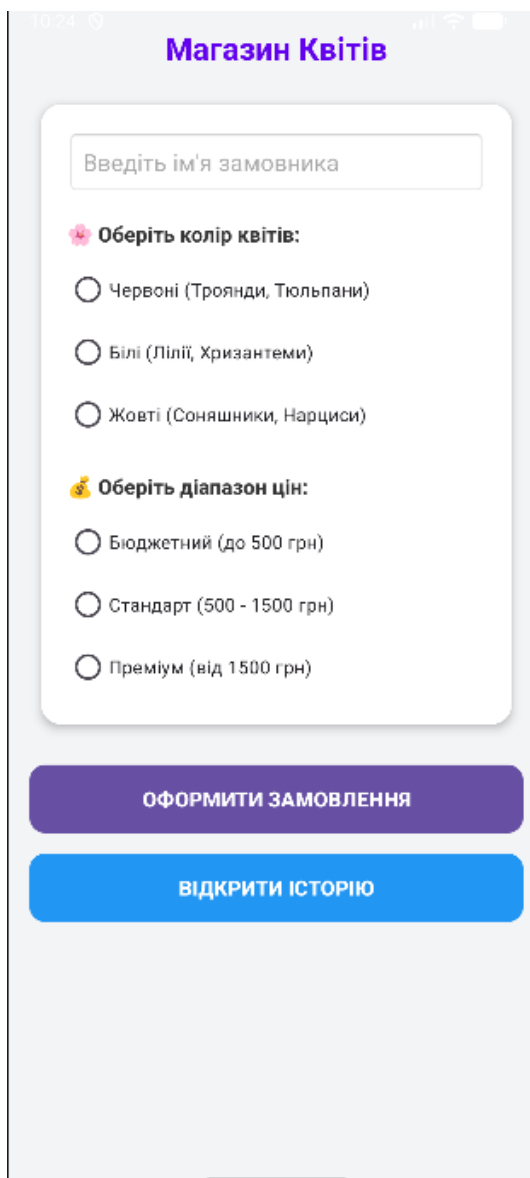


Рисунок 1 – Оновлений інтерфейс

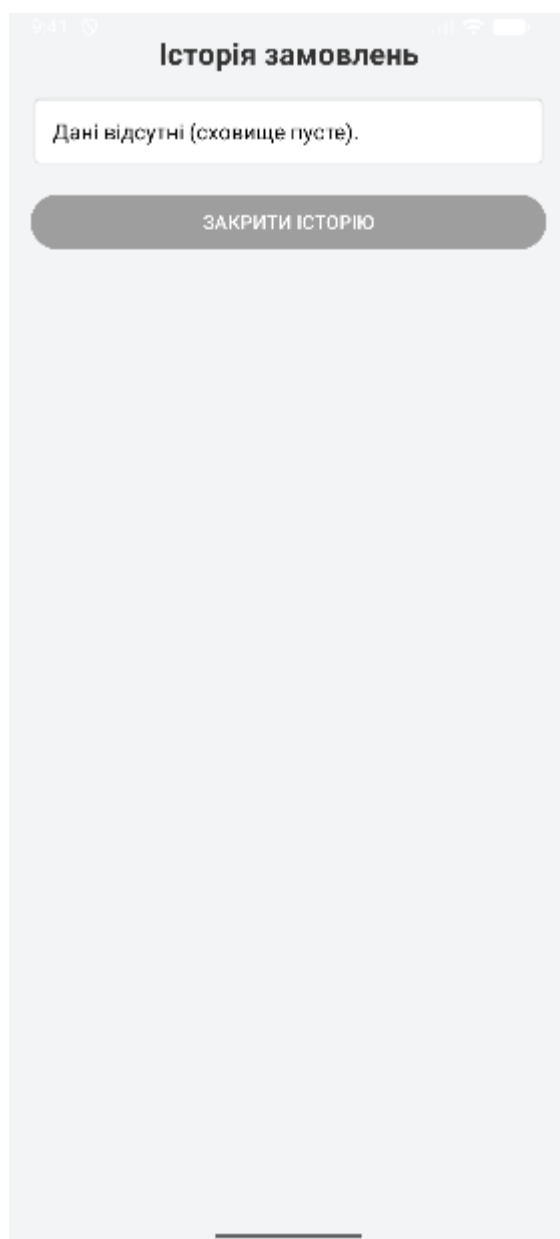


Рисунок 2 – Порожня історія

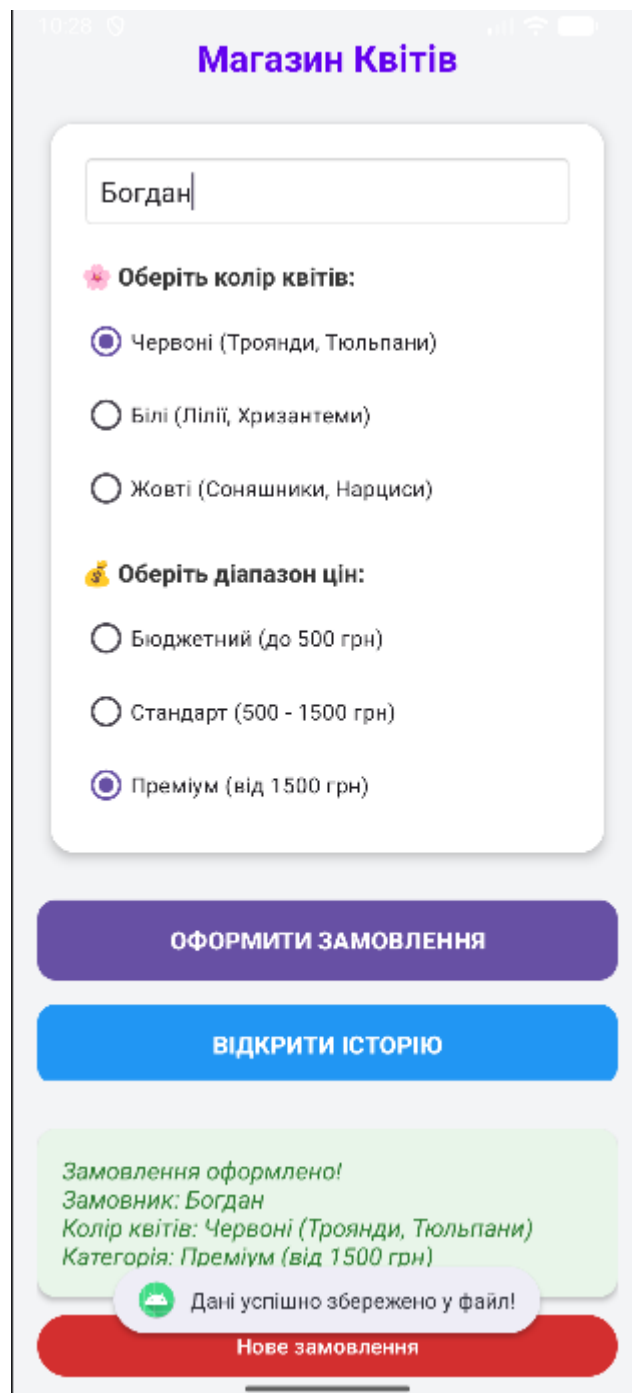


Рисунок 3 – Збереження даних



Рисунок 4 – Відображення історії

4. Висновок

Під час виконання лабораторної роботи було досліджено способи постійного збереження даних у середовищі Android. Практично реалізовано механізм запису текстових даних у локальний файл за допомогою `openFileOutput` (Internal Storage) та механізм зчитування за допомогою `openFileInput`. Було створено нову Activity для відображення збереженої історії та опрацьовано сценарій відсутності файлу зі збереженням стійкості роботи застосунку. Також вдосконалено UI/UX компоненти (перейменовано функціональну кнопку).

5. Контрольні питання

1. Опишіть як організована робота з налаштуваннями (даними у вигляді пари «ключ-значення»).

Робота з такими даними в Android організована через API `SharedPreferences`. Цей механізм дозволяють зберігати та отримувати невеликі обсяги примітивних даних (інти, булеві значення, рядки) у вигляді пар "ключ-значення". Дані зберігаються у XML-файлах у приватній директорії застосунку і автоматично завантажуються в пам'ять при старті.

2. Опишіть типи сховищ файлів та причини їх використання.

Internal Storage (Внутрішнє сховище): Приватне сховище застосунку. Файли недоступні іншим програмам і видаляються при видаленні застосунку. Використовується для конфіденційних даних.

- External Storage (Зовнішнє сховище): Спільне сховище (SD-карта або виділений розділ пам'яті). Файли доступні іншим програмам та користувачу. Використовується для медіафайлів, завантажень.
- Cache Storage (Кеш): Тимчасове сховище для прискорення роботи (наприклад, кешування картинок з мережі). Система може самостійно очистити його при нестачі пам'яті.

3. Опишіть процес роботи з файлами та файловою системою.

Процес базується на стандартних класах Java I/O (FileInputStream, FileOutputStream, File). Для доступу до внутрішнього сховища використовуються методи контексту `openFileInput(String name)` (для читання) та `openFileOutput(String name, int mode)` (для запису). Читання та запис зазвичай виконуються через потоки байтів або символів (наприклад, з використанням `InputStreamReader` та `BufferedReader`).

4. Опишіть процес роботи з базами даних SQLite за допомогою класу `SQLiteOpenHelper`, переваги та недоліки.

`SQLiteOpenHelper` — це допоміжний клас для управління створенням бази даних та керуванням її версіями. Розробник перевизначає методи `onCreate()` (для створення таблиць) та `onUpgrade()` (для міграції при зміні версії БД).

- Переваги: Вбудована в Android, не потребує окремого сервера, підтримує стандартний SQL.
- Недоліки: Багато шаблонного (boilerplate) коду, відсутність перевірки SQL-запитів на етапі компіляції, складність конвертації даних у Java/Kotlin об'єкти (потрібно працювати з `Cursor`).

5. Як відбувається робота з БД за допомогою бібліотеки Room, наведіть переваги та недоліки.

Room — це архітектурний компонент, який надає абстрактний шар над SQLite. Робота базується на анотаціях: `@Entity` (опис таблиць), `@Dao` (інтерфейси для доступу до даних) та `@Database` (сама БД).

- Переваги: Перевірка SQL-запитів на етапі компіляції, автоматична конвертація даних в об'єкти (ORM), мінімум шаблонного коду, нативна інтеграція з LiveData та Coroutines.
- Недоліки: Трохи вищий поріг входження, збільшення часу компіляції через процесинг анотацій.

6. Наведіть характеристики екранів мобільних пристроїв.

Основні характеристики:

- Фізичний розмір (Size): Діагональ екрана в дюймах (діляться на small, normal, large, xlarge).
- Роздільна здатність (Resolution): Загальна кількість фізичних пікселів на екрані (наприклад, 1080x1920).
- Щільність пікселів (Density - DPI): Кількість пікселів на дюйм площі екрана (ldpi, mdpi, hdpi, xhdpi, xxhdpi, xxxhdpi). Це критичний параметр для масштабування UI-елементів (використання одиниць dp та sp).
- Співвідношення сторін (Aspect Ratio): Пропорції ширини до висоти (наприклад, 16:9, 18:9).

7. Наведіть класифікацію та відмінності технологій (типів) екранів мобільних пристроїв.

LCD (TFT, IPS): Рідкокристалічні дисплеї з єдиним модулем підсвічування. IPS є покращеною версією з ширшими кутами огляду та точнішою передачею кольорів.

- OLED / AMOLED: Технологія на основі органічних світлодіодів. Кожен піксель випромінює світло самостійно. Відрізняються ідеальним чорним кольором (бо піксель просто вимикається), високою контрастністю та меншим енергоспоживанням при використанні темних тем.

8. Наведіть поняття та характеристику сенсорних екранів.

Сенсорний екран (Touchscreen) — це пристрій введення-виведення інформації, що реагує на дотики до нього. Головні характеристики: підтримка мультитач (розпізнавання кількох одночасних дотиків), час відгуку, чутливість до сили натискання та зносостійкість (наявність олеофобного покриття та захисного скла).

9. Наведіть загальну класифікацію сенсорних екранів.

Резистивні: Складаються з двох гнучких шарів. Реагують на механічний тиск (можна натискати чим завгодно, навіть у рукавичках). Застаріла технологія для смартфонів, не підтримує повноцінний мультитач.

- Ємнісні (Проекційно-ємнісні): Реагують на електричну ємність людського тіла (дотик пальцем). Є стандартом для сучасних мобільних пристроїв: довговічні, дуже чутливі, ідеально підтримують мультитач і жести.

10. Наведіть рекомендації щодо розробки інтерфейсів для сенсорних екранів.

1. Розмір клікабельних елементів (Touch targets) має бути не менше 48x48 dp, щоб користувачу було зручно влучати пальцем. 2. Забезпечення достатніх відступів між інтерактивними елементами для уникнення помилкових натискань. 3. Надання миттєвого візуального або тактильного (haptic) зворотного зв'язку при натисканні

кнопок. 4. Підтримка інтуїтивних жестів (свайпи, щипки). 5. Розташування найважливіших елементів керування у нижній частині екрана (у зоні досяжності великого пальця).